Compte rendu de conception UML

Projet Génie Logiciel

24 avril 2026

Ce document rassemble l'analyse fonctionnelle, statique et dynamique du projet. Il a vocation à présenter progressivement les choix de conception retenus, les diagrammes UML produits et les scénarios représentatifs du fonctionnement du système.

Table des matières

1 Introduction **3**

2 Axe Fonctionnel **3**
- 2.1 Présentation des fonctionnalités . . . 3
- 2.2 Diagramme de contexte statique . . . 3
- 2.3 Diagramme de cas d'usage . . . 4
- 2.4 Scénarios principaux . . . 5
  - 2.4.1 Charger un modèle système . . . 6
  - 2.4.2 Lancer/piloter la simulation . . . 7
  - 2.4.3 Importer est exécuter un scénario d'anomalies . . . 9

3 Axe Statique **10**
- 3.1 Vue d'ensemble . . . 10
- 3.2 Paquetage Generic Model . . . 10
- 3.3 Paquetage Data Management . . . 11
- 3.4 Paquetage Construction . . . 12
- 3.5 Paquetage Simulation . . . 13
- 3.6 Paquetage Control . . . 14
- 3.7 Paquetage GUI . . . 15

4 Axe Dynamique **16**

5 Partie IA **17**
- 5.1 Jeu de donnees de 3000 lignes . . . 17
- 5.2 Jeu de donnees de 10 000 lignes . . . 18

6 Conclusion **18**
- 6.1 Gestion d'équipe . . . 18
- 6.2 Difficultés rencontrées . . . 18
- 6.3 Perspectives d'avenir . . . 19

1 Introduction

Pour des systèmes complexes, on est souvent confronté à des difficultés pour collecter des données de qualité pour entraîner des modèles de maintenance. D'où l'idée de GADMAPS, Générateur d'Anomalies et de Données pour la Maintenance Prédictive des Systèmes, un logiciel générique qui nous permet de simuler le fonctionnement de n'importe quel système technique peu importe sa complexité et le nombre de ses sous-systèmes.

Ce compte rendu de conception UML a pour objectif de présenter les différentes étapes de l'analyse et de la conception du projet GADMAPS. Nous allons d'abord présenter les fonctionnalités principales du système, puis nous détaillerons la structure statique du projet en termes de paquetages et de classes, puis nous passerons un peu moins en détail sur l'axe dynamique.

Finalement, une conclusion qui comportera une partie sur les difficultés rencontrées, la gestion du projet et les perspectives d'amélioration.

2 Axe Fonctionnel

2.1 Présentation des fonctionnalités

Cette partie résumera les services rendus à l'utilisateur, en restant au niveau des objectifs métier plutôt qu'au niveau des détails d'implémentation.

GADMAPS offre les fonctionnalités suivantes :
- Charger un modèle de système à partir d'une source de données.
- Configurer les paramètres de simulation, y compris les types d'anomalies à injecter et les capteurs à activer.
- Lancer la simulation et observer les résultats en temps réel.
- Exporter les données de simulation.
- Gérer les scénarios d'anomalies.
- Visualiser les données de simulation à travers une interface détaillée.

2.2 Diagramme de contexte statique

La figure 1 présente le système dans son contexte statique et met en évidence les acteurs principaux et secondaires qui interagissent avec lui.

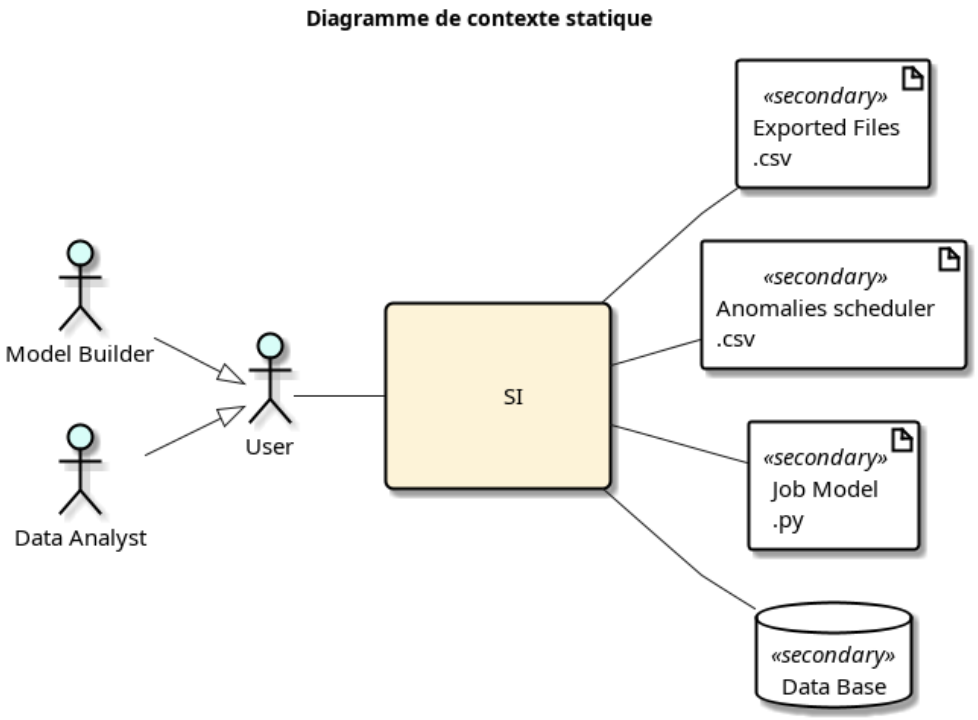


FIGURE 1 – Diagramme de contexte statique du SI

Ce diagramme nous permet de determiner les différents acteurs qui interagissent avec le système :

- **User** : L'utilisateur est l'acteur principal pour lequel nous avons conçu le système. On peut imaginer deux rôles pour cet acteur :
  - **Data Scientist** : qui utilise les systèmess modelisés par GADMAPS pour recupérer les données utiles à la maintenance.
  - **Model Creator** : qui utilise GADMAPS pour créer des modèles et les visualiser.
- **Exported Files** : les fichiers qui sont exportés à chaque fin de simulation.
- **Job Model** : ce sont les fichiers python genérés par le Model Creator qui définissent le modèle, ses sous-système et la possibilité de placer des capteurs.
- **Anomalies Scheduler** : c'est un fichier qui contient les scénarios d'anomalies à injecter pendant la simulation.
- **Data Base** : c'est un acteur secondaire qu'on a imaginé mais pas implementé, qui pourrait être utilisé pour stocker les données de simulation d'une manière sécurisée, ou même pour faire l'authentification des utilisateurs.

2.3 Diagramme de cas d'usage

D'après les fonctionnalités présentées et les acteurs identifiés, on peut construire le diagramme de cas d'usage du système présenté dans la figure 2.

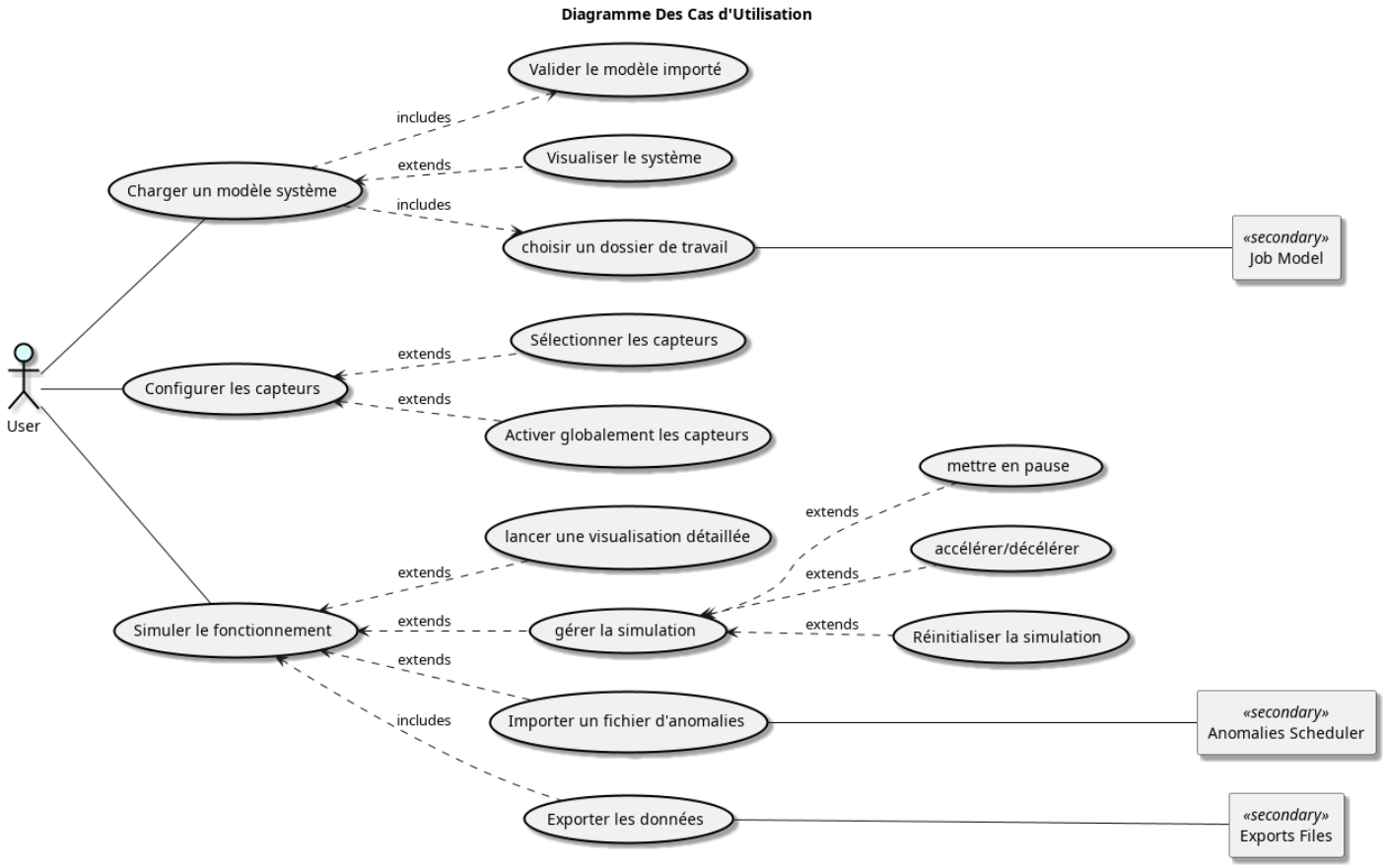


FIGURE 2 – Diagramme de cas d'usage

2.4 Scénarios principaux

On a choisi de présenter un scénario textuel pour illustrer les étapes nécessaires pour charger un modèle système. Et trois diagrammes de séquence pour illustrer :

- Le chargement d'un modèle système.
- Lancer/piloter la simulation.
- Importer et exécuter un scénario d'anomalies.

2.4.1 Charger un modèle système

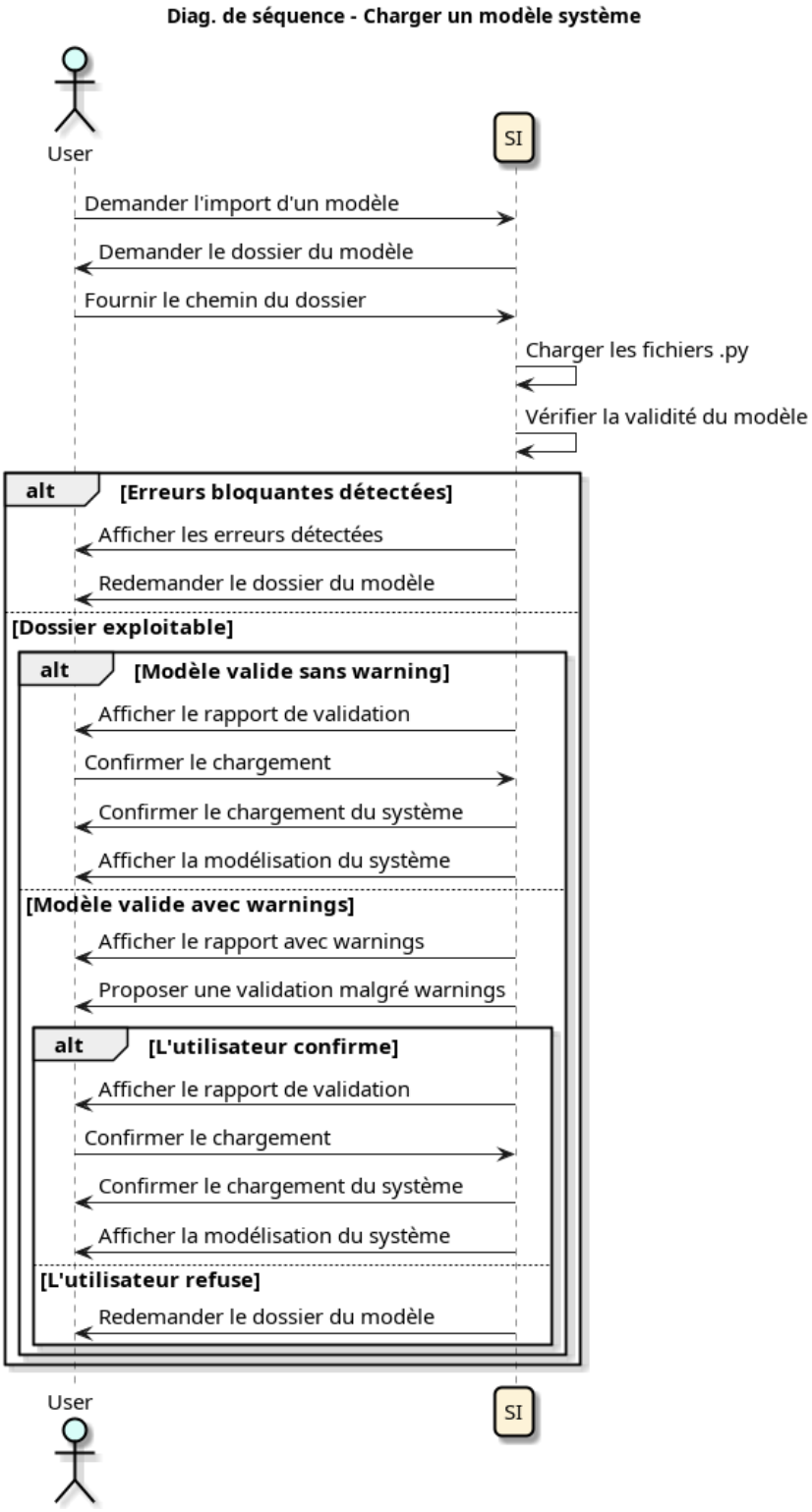


FIGURE 3 – Diagramme de séquence pour le chargement d'un modèle système

2.4.2 Lancer/piloter la simulation

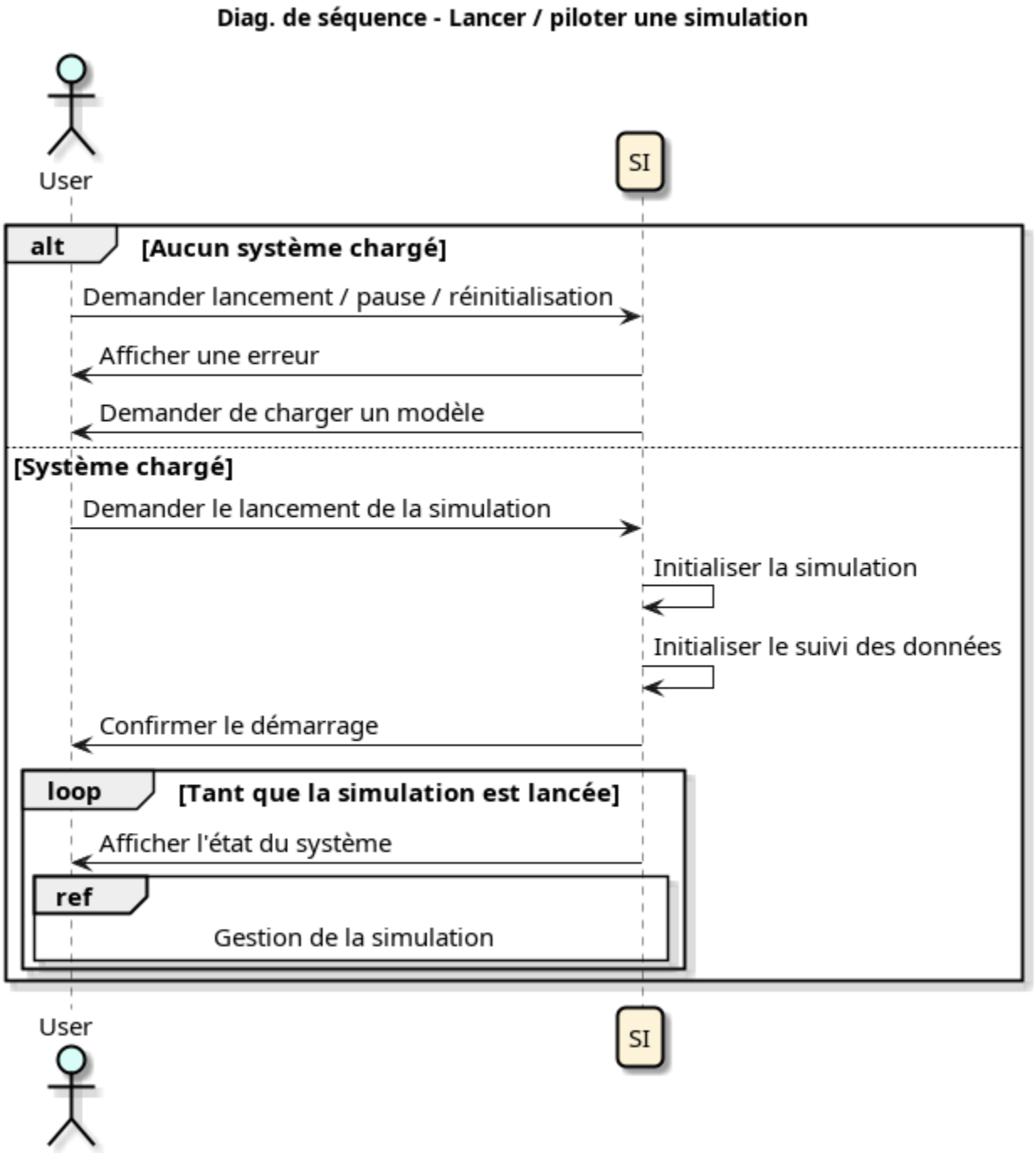


FIGURE 4 – Diagramme de séquence pour le lancement/pilotage d'une simulation

Ce diagramme contient une référence au cas d'usage "Gestion de la simulation" qu'on peut représenter par le diagramme de séquence présenté dans

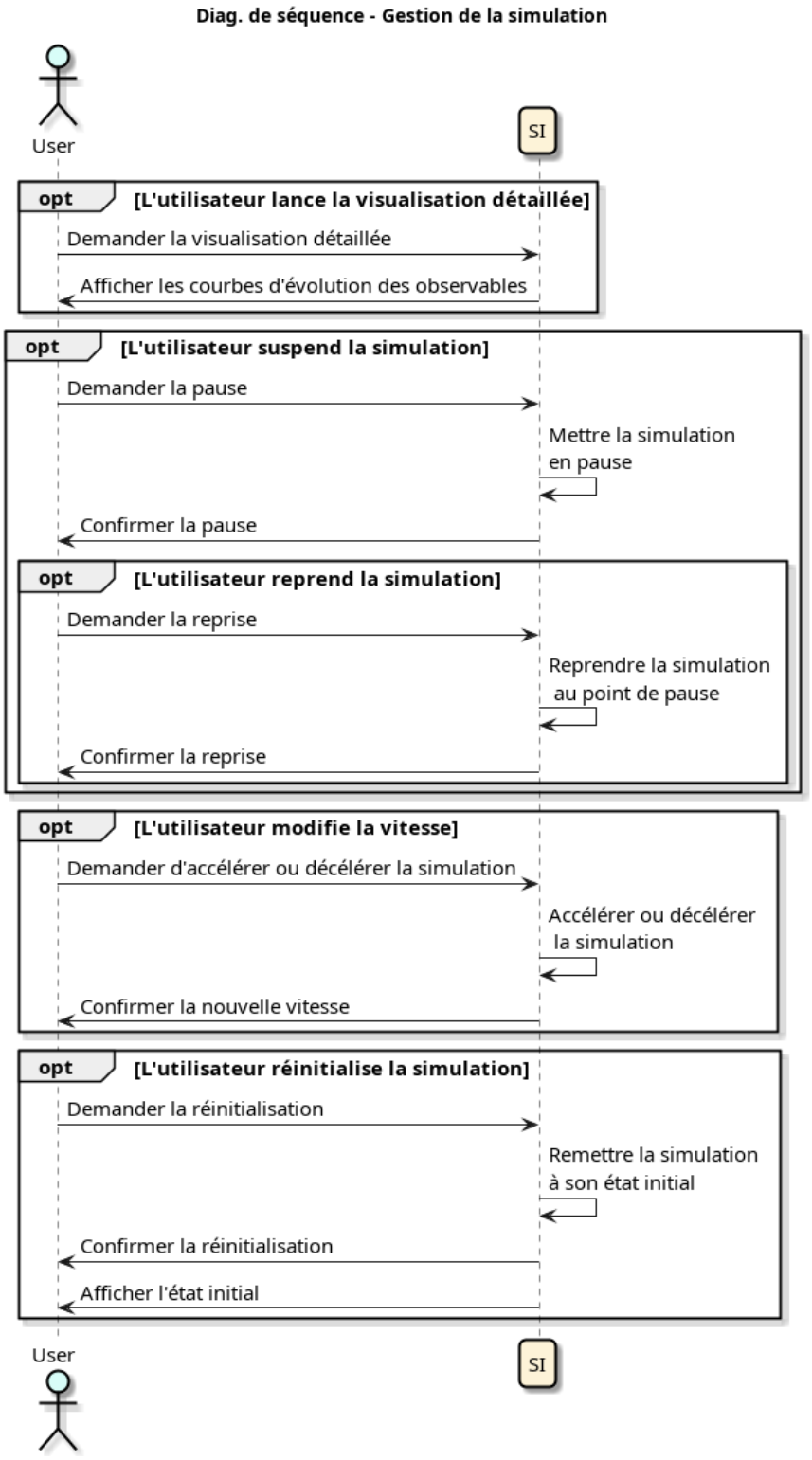


FIGURE 5 – Diagramme de séquence pour la gestion d'une simulation

2.4.3 Importer est exécuter un scénario d'anomalies

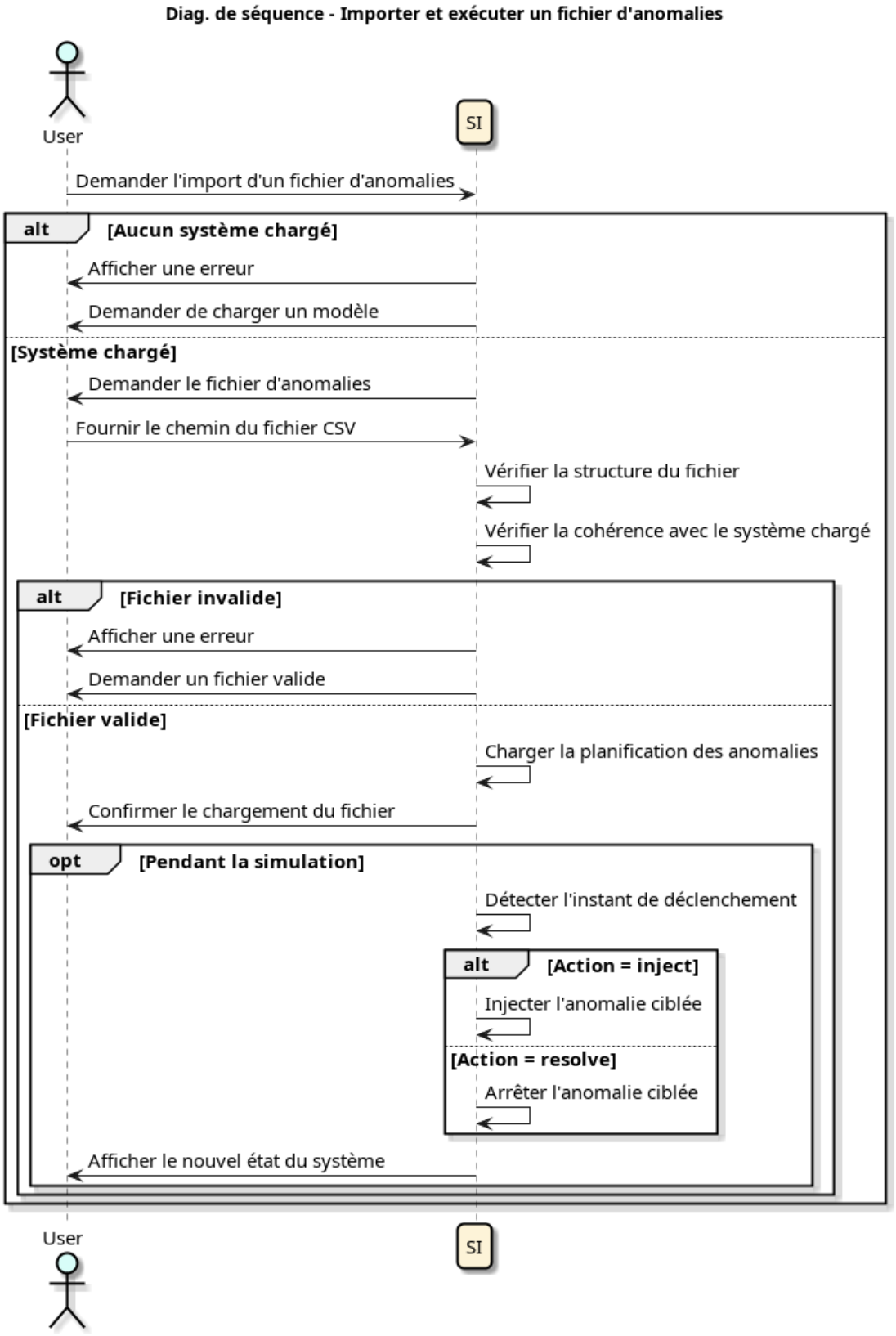


FIGURE 6 – Diagramme de séquence pour l'import et l'exécution d'un scénario d'anomalies

3 Axe Statique

3.1 Vue d'ensemble

Une fois les fonctionnalités du système définies avec les scénarios et les diagrammes de séquence. On peut s'interesser à la structure statique du projet, c'est à dire comment les différentes classes sont organisées en paquetages et comment elles interagissent entre elles.

On a choisi une architecture MVC dans le projet, avec une programmation orientée objet et modulaire. Ce choix a influé sur la manière dont nous avons decouper le projet en paquetages :

- **Generic Model**.
- **GUI**.
- **Control**.
- **Data Management**.
- **Construction**.
- **Simulation**.

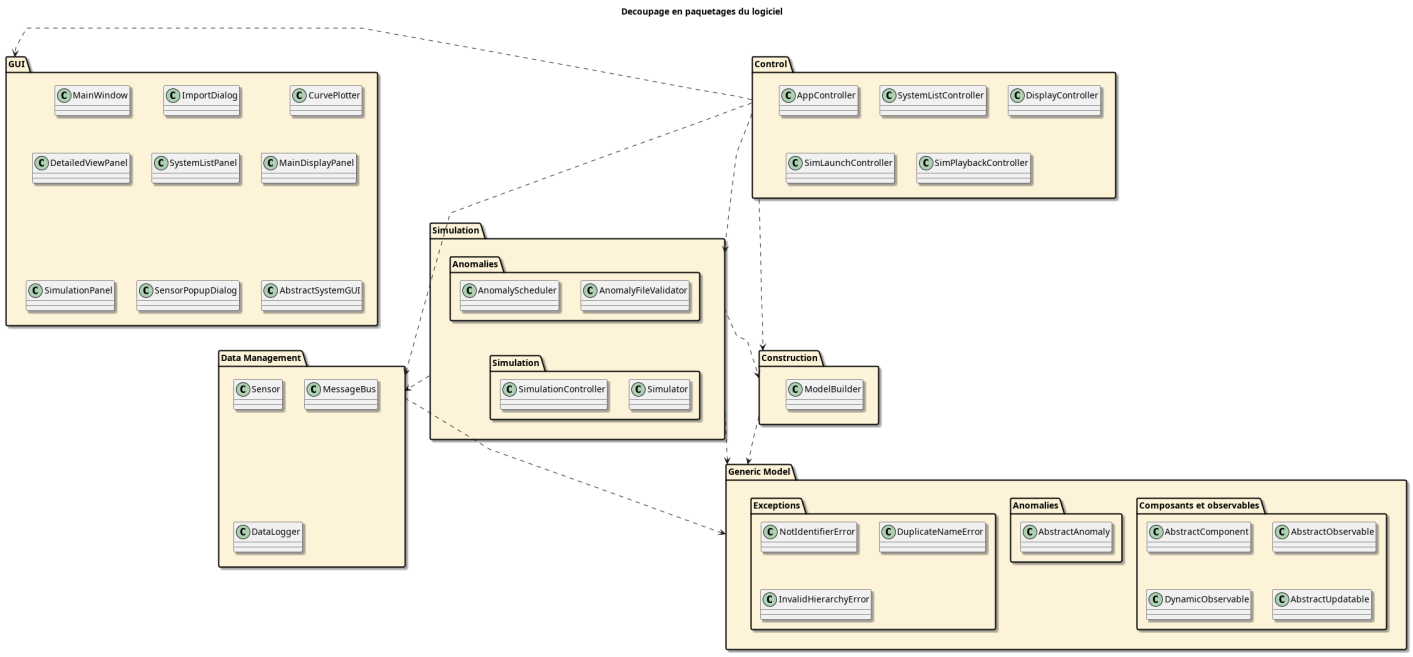


FIGURE 7 – Diagramme de découpage en paquetages des classes

3.2 Paquetage Generic Model

sert à définir le noyau du système, elle fournit les classes abstraites qui permettent de représenter n'importe quel système technique, avec ses composants :

- **AbstractComponent** : qui représente un composant du système avec ses enfans, ses observables et ses anomalies.
- **AbstractObservable** : qui représente une valeur mesurable du système.
- **DynamicObservable** : c'est la règle qui permet de calculer la valeur mesurable.

— **AbstractAnomaly** : elle définit le comportement d'une anomalie injectable sur une composante.
On introduit donc le diagramme suivant :

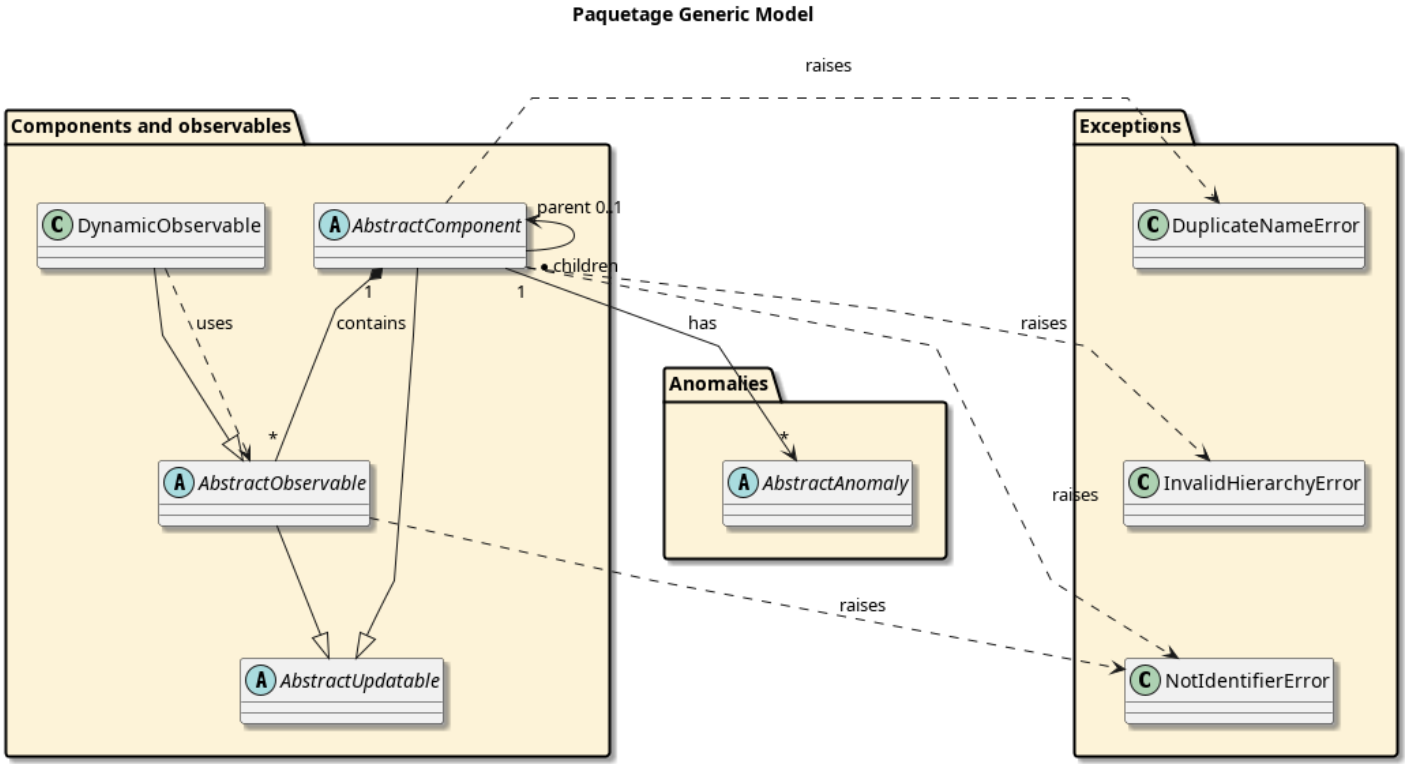


FIGURE 8 – Diagramme de classes du paquetage Generic Model

3.3 Paquetage Data Management

C'est la partie du système qui gère les données de simulation, dans notre projet on s'en sert de :

— **Sensor** pour lire les valeurs des observables.
— **MessageBus** pour trasnmettre ces données via un bus de message.
— **DataLogger** pour enregistrer les données dans les fichiers csv.

Dans notre implémentation, les capteurs publient leurs mesures sur des canaux construits à partir de la hiérarchie du système. Cela permet d'éviter les collisions de noms entre deux observables différentes.

Le logger enregistre uniquement les capteurs activés par l'utilisateur, avec leur horodatage de simulation, puis écrit les données dans un fichier csv.

On en deduit le diagramme suivant :

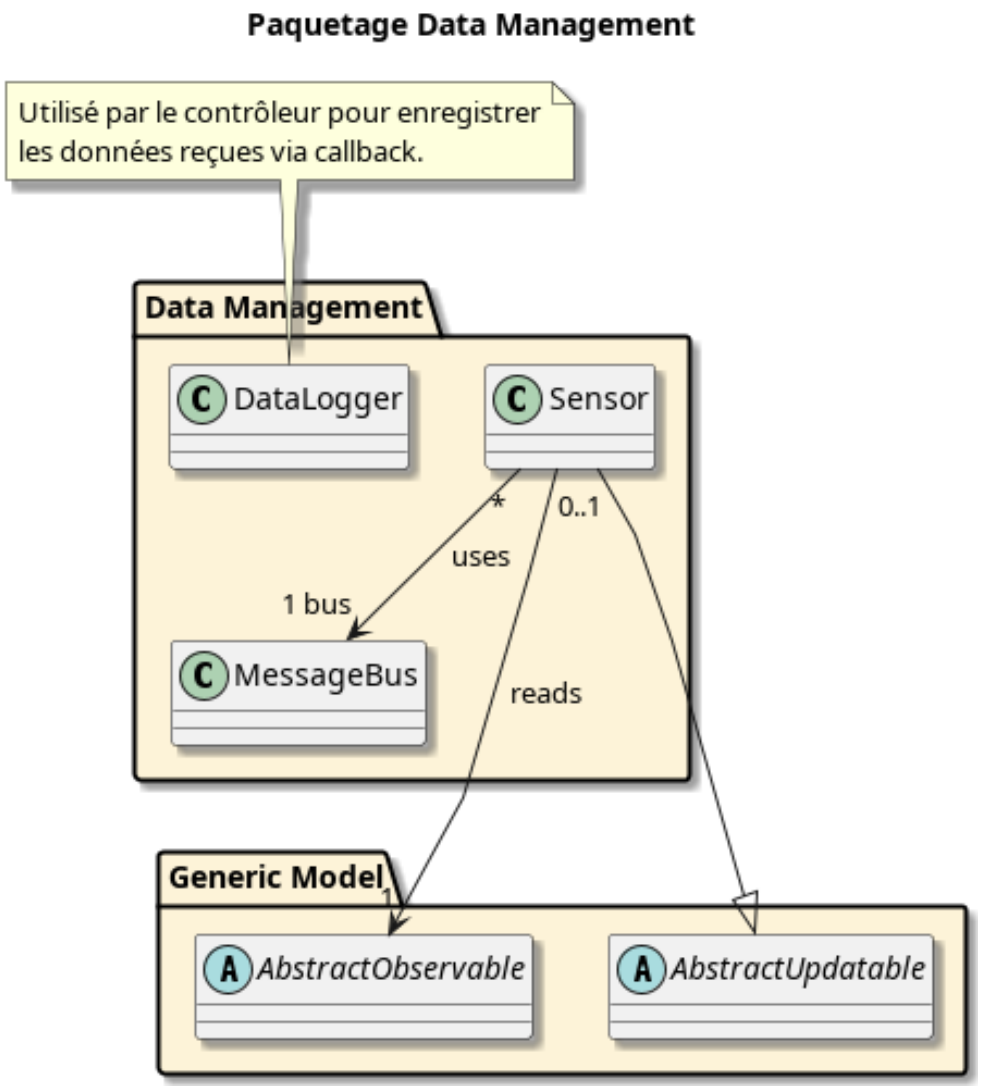


FIGURE 9 – Diagramme de classes du paquetage Data Management

3.4 Paquetage Construction

le paquetage correspond à la partie qui construit le système technique à partir des fichiers du modèle metier. Son rôle est de charger les fichiers python, inspecter les classes, ensuite constuire le model avec **Model Builder** après avoir vérifié la conformité du modèle.

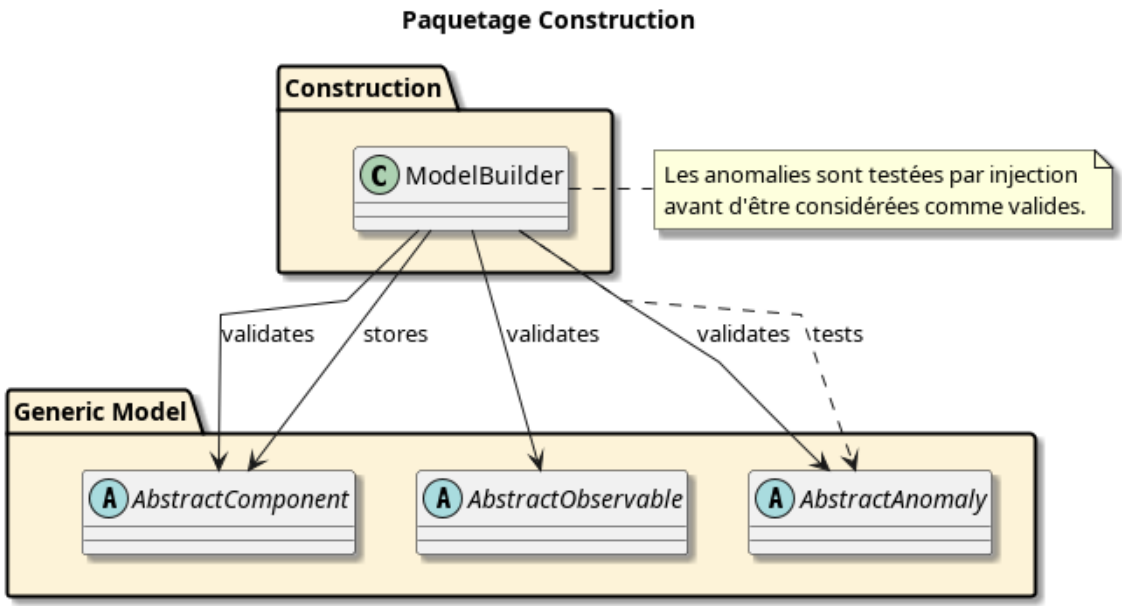


FIGURE 10 – Diagramme de classes du paquetage Construction

3.5 Paquetage Simulation

Une fois le système technique construit, on peut lancer la simulation, c'est le rôle du paquetage Simulation. Son rôle est de faire vivre le système principalement avec :

- **Simulator** : qui fait tourner la simulation.
- **Anomalies Scheduler** : qui déclenche les anomalies.
- **Anomaly File Validator** : vérifie la conformité du fichier d'anomalies.

Lorsqu'un système est sélectionné dans l'application, il est initialisé à t=0 pour permettre à l'utilisateur de configurer ses capteurs avant le lancement. Ensuite la simulation avance pas à pas, et le reset permet de recréer proprement le système depuis son état initial.

Le validateur de fichier d'anomalies ne vérifie pas seulement la présence des colonnes attendues, il contrôle aussi la cohérence des temps, l'existence du composant ciblé, l'existence de l'anomalie et l'action demandée.

On peut donc présenter le diagramme suivant :

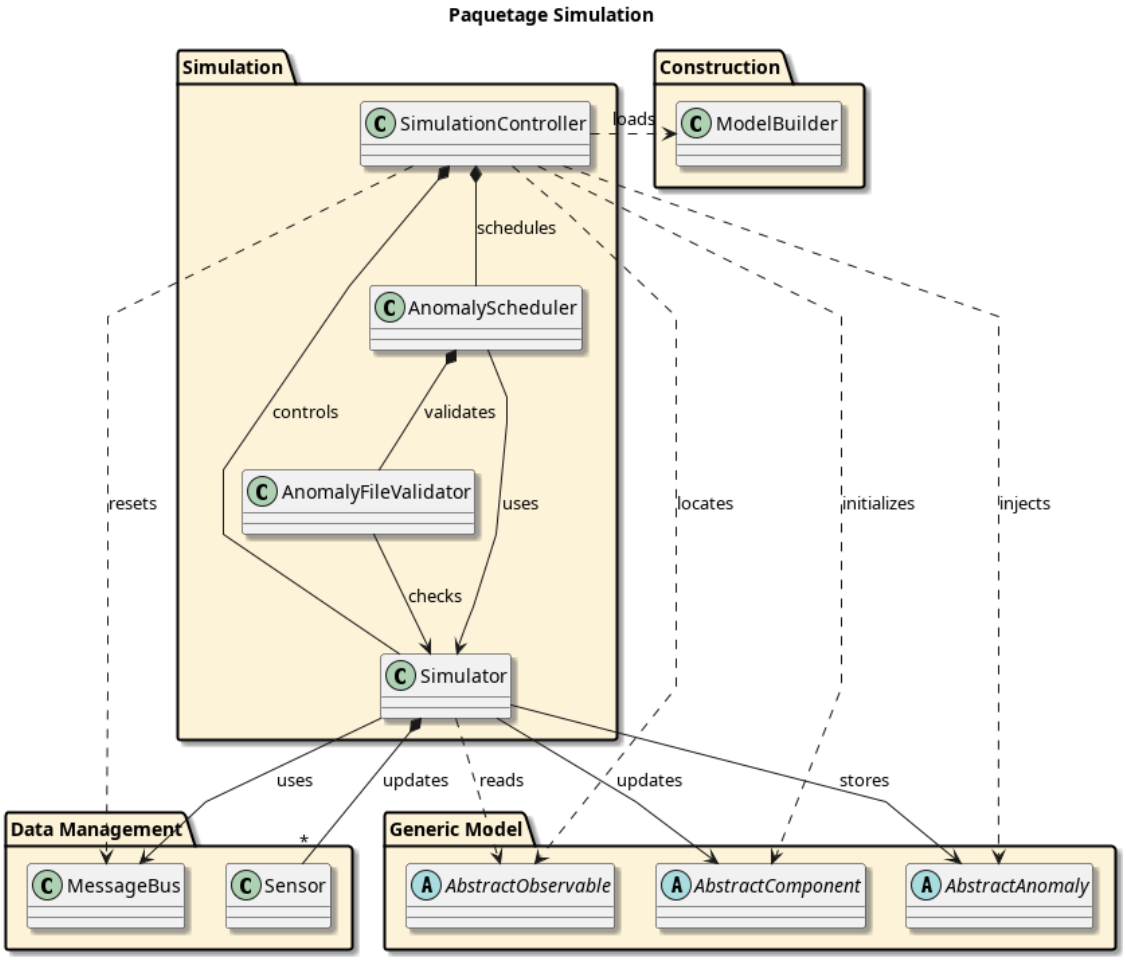


FIGURE 11 – Diagramme de classes du paquetage Simulation

3.6 Paquetage Control

a pour rôle d'orchestrer les différentes interactions entre la vue et le modèle, il fait le lien entre l'utilisateur, l'interface graphique et le backend par :

- **App Controller** : c'est le contrôleur principal qui coordonne l'ensemble.
- Et puis, des contrôleurs pour **l'interface** :
  - System List Controller.
  - Display Controller.
  - Sim Launch Controller.
- Des controleurs pour **le backend** :
  - Simulator Controller.

Le contrôleur principal gère aussi la liste des systèmes importés, la sélection du système courant, le lancement de la boucle de simulation et la mise à jour des graphes et du journal d'exécution.

Le diagramme suivant les différents liens entre ces classes et les classes du système.

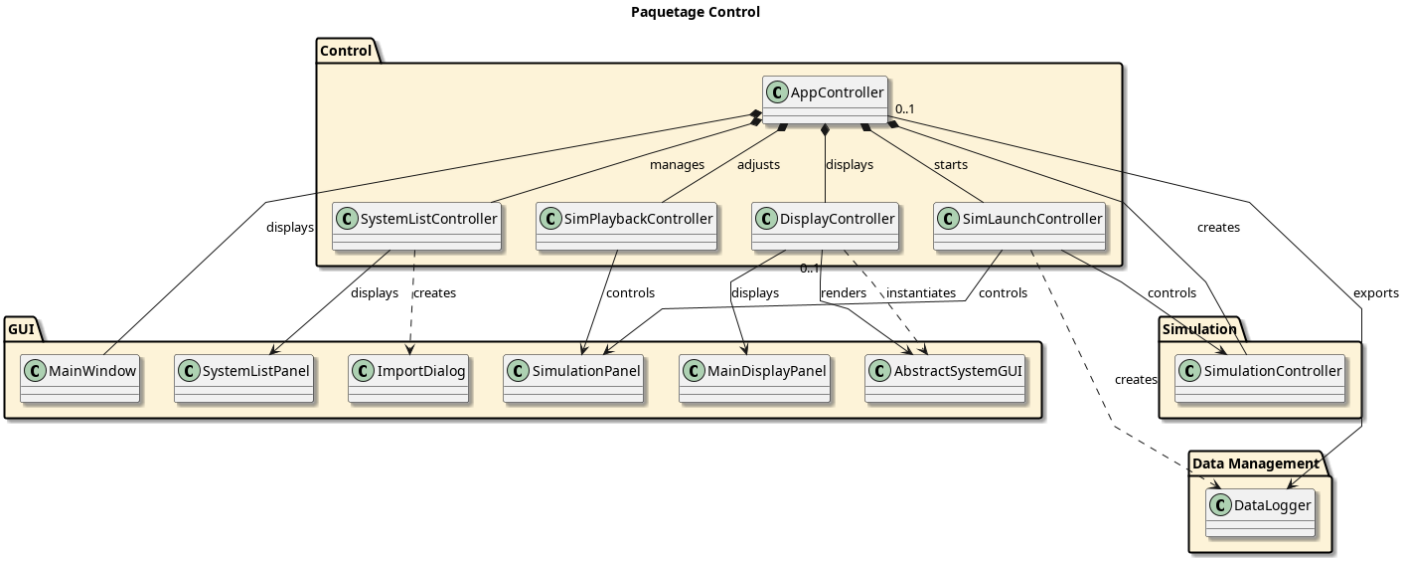


FIGURE 12 – Diagramme de classes du paquetage Control

3.7 Paquetage GUI

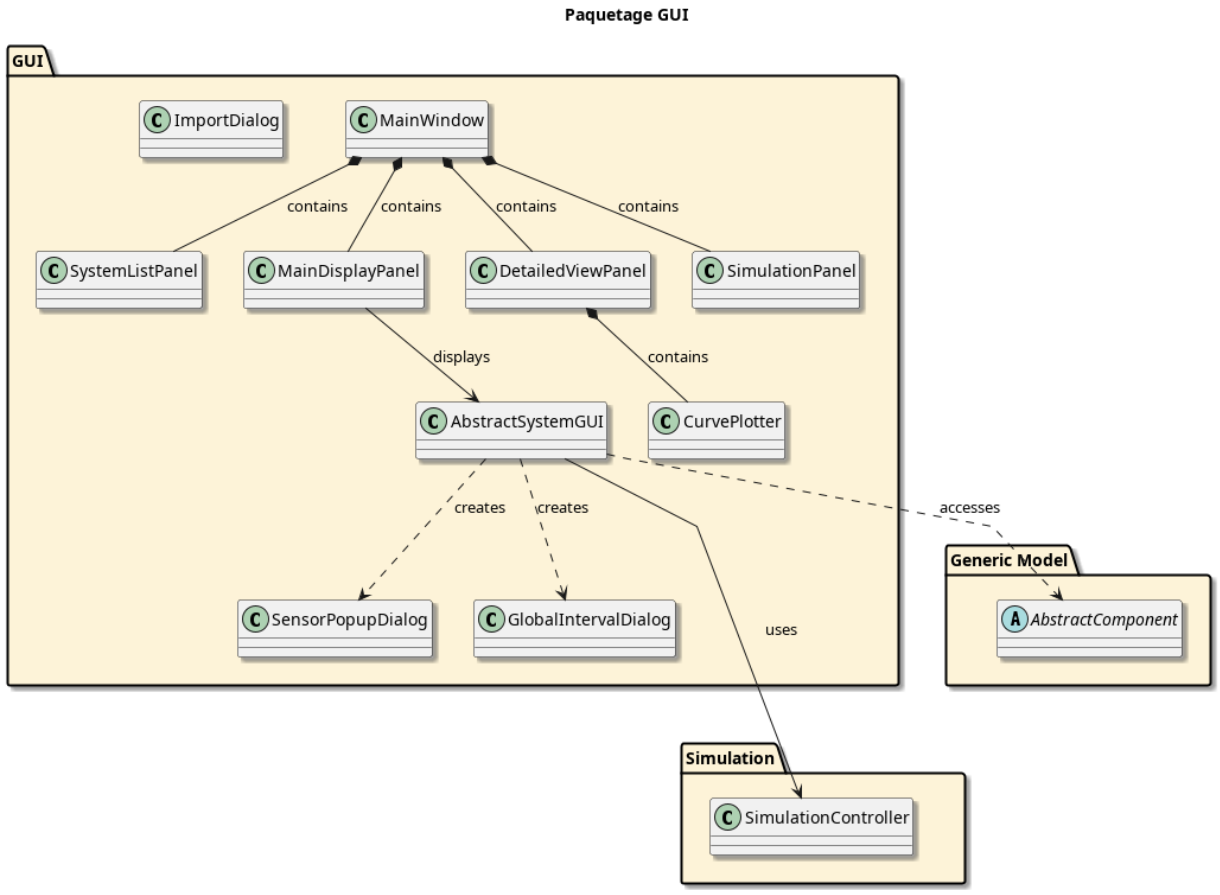


FIGURE 13 – Diagramme de classes du paquetage GUI

présente l'aspect graphique du projet, il contient les classes qui permettent de construire l'IHM. On peut le représenter par le diagramme suivant.

L'interface permet d'importer un dossier système, d'afficher un bilan d'introspection, puis de bloquer l'import si des erreurs métier sont détectées. Elle permet aussi de configurer les capteurs observables, d'activer tous les capteurs en une seule fois avec un intervalle commun, d'afficher un journal textuel des mesures et d'ouvrir une vue détaillée des courbes.

Une fois la simulation démarrée, la modification des capteurs est bloquée jusqu'à la réinitialisation. Après un reset, la configuration précédente peut être réappliquée automatiquement au nouveau backend.

4 Axe Dynamique

Une tentative de création de l'axe dynamique a été faite vu qu'on avait pas eu des cours sur les diagrammes dynamiques. On a choisi de faire un diagramme pour "l'UC lancer une simulation".

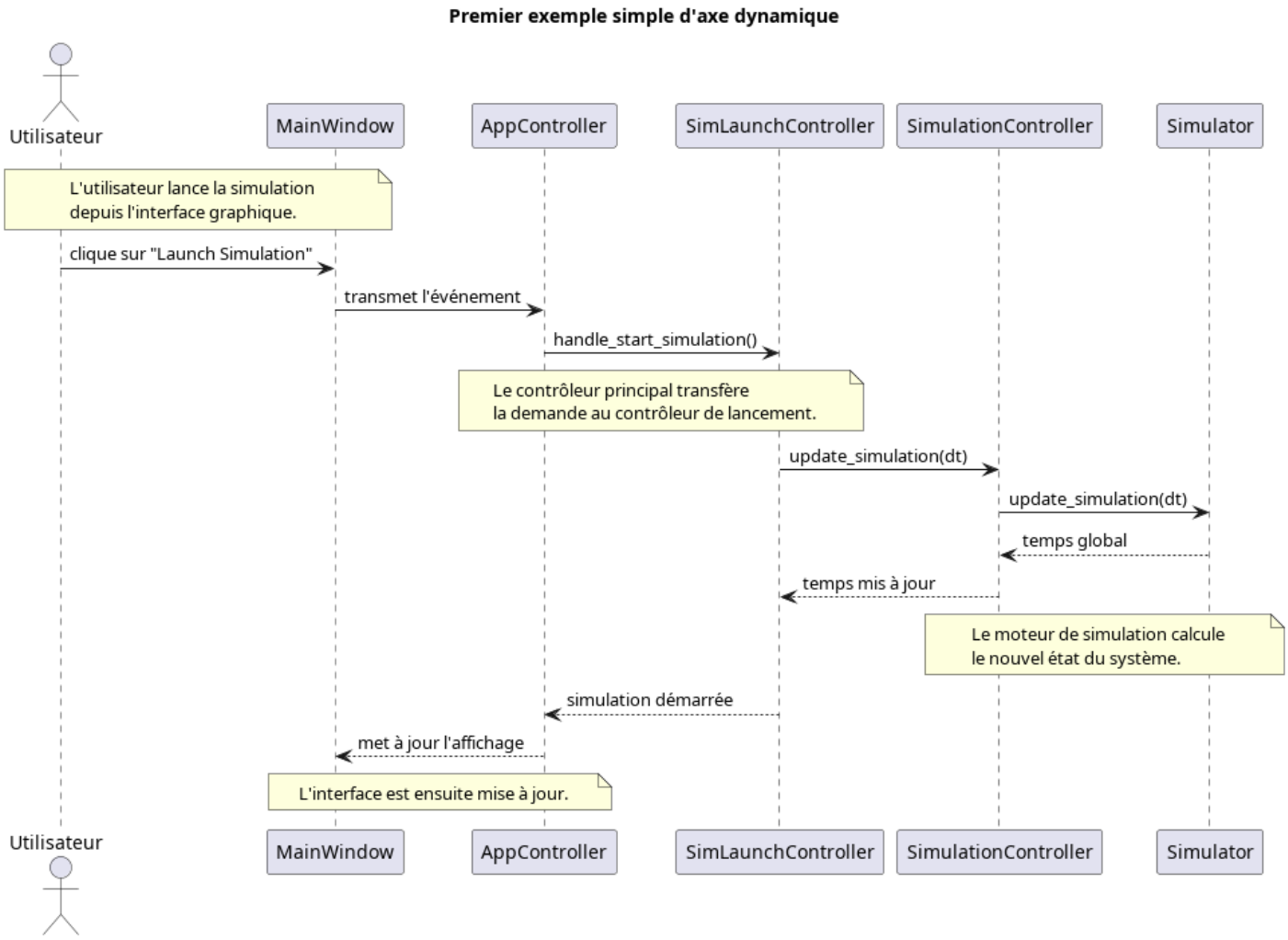


FIGURE 14 – Diagramme dynamique pour l'UC "Lancer une simulation"

5 Partie IA

La partie IA était optionnelle dans le sujet, mais on a quand même essayé de faire une première implémentation d'un modèle d'IA discriminatif pour prédire les anomalies à partir des données de simulation.

Pour finaliser le but du logiciel qui est de générer des données pour entraîner des modèles de maintenance prédictive.

La maintenance se fait a partir des etapaes suivantes :
- Generer des données avec GADMAPS.
- Preparer les données.
- Normaliser les données (normalisation globale sur tous les fichiers)
- Entraîner le modèle d'IA.
- Tester le modèle avec des données anormales.
- Lisser les résultats pour stabiliser la détection et choisir le seuil de détection empiriquement.
- Evaluer les performances du modèle avec les anomalies réellement injectées.

5.1 Jeu de donnees de 3000 lignes

Un premier test a été fait avec un jeu de données de 3000 lignes avec 5 anomalies injectées. Cette figure montre que le modèle a réussi à détecter les anomalies.

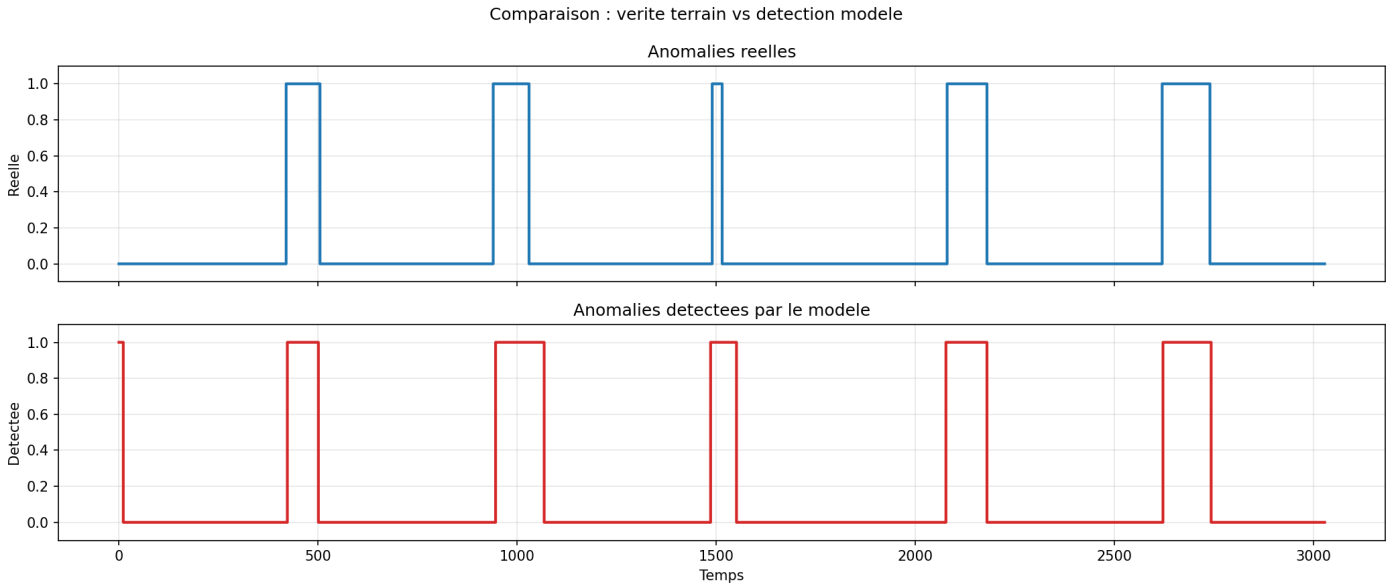


FIGURE 15 – Résultats du modèle d'IA sur un jeu de données de 3000 lignes

5.2 Jeu de donnees de 10 000 lignes

Un deuxième test a été fait avec un jeu de données de 10 000 lignes avec 10 anomalies injectées. Cette figure montre que le modèle a réussi à détecter les anomalies mais cette fois fallait rajouter plus de données d'entainement qui depassent les 6000 lignes pour que le modèle puisse apprendre les comportements normaux au delas des 6000 premières lignes.

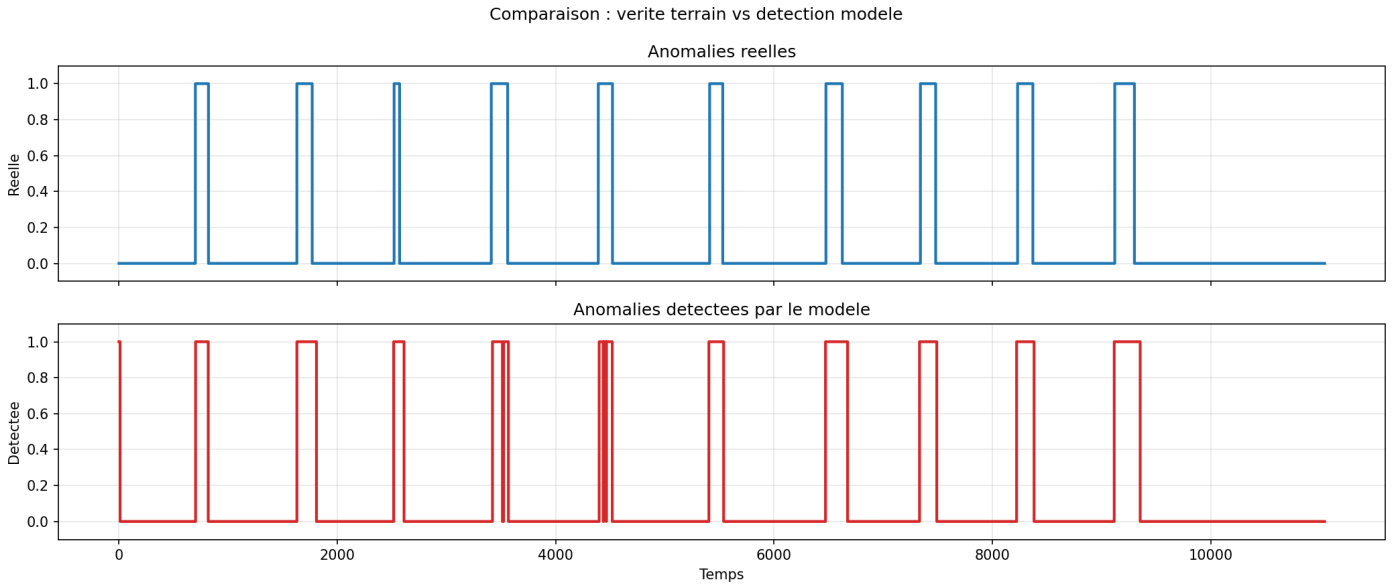


FIGURE 16 – Résultats du modèle d'IA sur un jeu de données de 10 000 lignes

6 Conclusion

6.1 Gestion d'équipe

Nous avons essayé de suivre une méthodologie agile pour la gestion de notre projet, une méthodologie incrementale et itérative, qui nous a permis d'avancer sur plusieurs axes du projet en parallèle, communiquer régulièrement pour faire le point sur l'avancement et les difficultés rencontrées chaque semaine en TP ou en dehors.

GitHub nous a permis de collaborer efficacement, de partager le code et fusionner les différents modules du projet, en suivant notre emploi du temps qu'on a défini et en respectant les deadlines.

6.2 Difficultés rencontrées

Nous avons rencontré plusieurs difficultés au cours de ce projet, notamment :

- La complexité du projet et la nécessité de faire des choix de conception.
- La généricité du logiciel qui nous a obligé à faire le choix d'introspection.
- Le manque de temps pour implémenter toutes les fonctionnalités prévues.

— La difficulté de faire l'axe dynamique vu qu'on n'avait pas eu de cours sur les diagrammes dynamiques.

6.3 Perspectives d'avenir

Pour améliorer ce projet, on pourrait envisager les perspectives suivantes :

— Ajouter une base de données pour stocker les données de simulation de manière sécurisée
— Améliorer la conception avec l'axe dynamique.
— Ajout de nouvelles fonctionnalités, comme integrer le modèle IA dans l'interface graphique du logiciel.
— Améliorer l'interface graphique et le rendre plus agréable à l'utilisateur.
— Faire tous les tests unitaires pour assurer la qualité du code.